

The GDD Standard

Format, Rules, and Writing Guide for Game Design Documents

Repo: Multi-agents-class · **Course:** Multi-Agent AI for Game Development (ELVTR)

Status: Living document — v1.0, 22 July 2026

This is the house standard for every GDD written in this repo. It merges industry practice (Tim Ryan's classic GDD anatomy, Stone Librande's one-page visual designs, the modern "lean/living document" school) with the course's own rules from Class 03 ("cut the wishes, keep the specifications") and the grading rubrics of Assignments #1 and #2.

Part I — What a GDD Is (and Is Not)

A GDD is a **buildable specification and an alignment tool**. Its one job: after reading it, a stranger can (a) describe your game to someone else, and (b) start building it without asking you what you meant.

A GDD is	A GDD is not
A spec: exact mechanics, numbers, formats	A pitch deck or marketing copy
Written from the player's point of view	A description of what the system computes
Short enough to maintain (living document)	A 100-page novel nobody updates
Testable: every claim can be checked	A wish list ("the AI will make it fun")
Specific to <i>this</i> game	A template filled with genre boilerplate

The one-sentence test for every line you write: *Could a reviewer prove this sentence wrong by playing the build?* If nothing could ever falsify it ("the game is immersive"), it's not design — delete it.

Part II — The Ten Golden Rules

1. **Specs, not wishes.** ■ "The AI will generate interesting dialogue."
■ "The Content Agent generates NPC dialogue in JSON: speaker_id, tone (one of 5), max 40 words per line."
2. **Player-facing first.** Describe what the player sees *and does*, then how the system produces it. Never the reverse.
3. **Numbers over adjectives.** "Fast" is an opinion; " ≤ 3 s reply latency" is a spec. "Hard" is an opinion; "30 actions per night" is a spec.
4. **Name everything.** The game, every mechanic, every agent, every resource. Named things can be discussed; "the system" cannot.
5. **Every agent gets three things:** a name, a one-sentence development

role, and an output format. No exceptions.

6. **Revision makes it shorter, not longer.** More specific $\neq$ more words.

Cut anything that doesn't change what gets built.

7. **Present tense, active voice.** "The player selects a suspect" — not "the player will be able to select" or "suspects can be selected."

8. **One idea per paragraph, $\leq$ 4 sentences.** If a paragraph needs a "furthermore," it's two paragraphs.

9. **Acknowledge at least one real constraint** (cost, latency, time, skill) and show how the design absorbs it. A GDD with no constraints is fiction.

10. **Read it aloud.** If a section sounds vague spoken, it *is* vague (Class 03 rule). If a table can replace a paragraph, use the table.

Part III — Required Sections, Bar by Bar

The house skeleton (order matters — it's the order a stranger needs the information in):

1. Title & Metadata
 2. Executive Summary
 3. Game Mechanics (core loop)
 4. Player Experience (what the player sees)
 5. AI / Agent Architecture
 6. Technical Strategy (stack, token budget, constraints)
 7. Scope & Development Plan
- [Optional appendices: narrative bible, art direction, audio, balancing tables]
-

1. Title & Metadata

Purpose: Identify the document at a glance.

Must contain: Game title (real title, not "My Capstone Game"), document type + version ("GDD — First Draft v1.2"), author, date, course/assignment if applicable.

Must NOT contain: Taglines, quotes, cover art essays.

Length: Half a page maximum.

2. Executive Summary

Purpose: A stranger reads only this and can still describe your game.

Must contain — all five, no substitutes:

- **Concept in $\leq$ 3 sentences** — who the player is, what they do, where.
- **Win condition and loss condition(s)** — stated explicitly, in bold if you like. This is the #1 thing graders and reviewers look for.
- **Platform & format** — "text-based browser game," "Unity 2D, PC."

- **Team & timeline reality** — "one developer, 4 weeks."
- **An elevator pitch by comparison** — "*Her Story* meets *Ace Attorney*."

One line. Comparisons transmit more design information per word than any other sentence in the document.

Must NOT contain: Market analysis, revenue projections, "target audience is 18–35 gamers," lore dumps, adjective stacks ("a thrilling, immersive, unforgettable experience").

Length: Half a page. If it's a full page, you're pitching, not specifying.

■ **Bad:** "This revolutionary game combines AI and storytelling to create an immersive detective experience unlike anything before. Players will love uncovering its endless mysteries."

(No player action, no win condition, three unfalsifiable adjectives.)

■ **Good:** "The player is a detective with until dawn (30 actions) to interrogate five AI-driven suspects about a locked-room murder. They win by accusing the correct suspect with at least 2 pieces of supporting evidence; they lose by accusing wrongly or running out the clock."

(Role, verb, resource, win, loss — all in two sentences.)

3. Game Mechanics (Core Loop)

Purpose: Define what the player does, in order, repeatedly — and why the loop doesn't degenerate.

Must contain:

- **The core loop as a numbered list** (3–6 steps). If you can't write it as a numbered list, you don't have a loop yet.
- **The cost/economy of actions** — what is scarce (time, actions, gold, health) and exactly what each verb costs.
- **The difficulty/pressure system** — what stops the player from winning by doing the obvious thing forever.
- **Failure and edge cases** — what happens on wrong input, wasted actions, stalemates.
- **A "why it's a game" paragraph** — one paragraph proving the player can't brute-force it. (For AI games: why it isn't just a chatbot.)

Must NOT contain: Implementation details (API calls, class names), narrative backstory, feature lists with no connection to the loop ("also there will be crafting").

Length: 1–2 pages. This section and §5 carry the document.

■ **Bad:** "Players interrogate suspects to solve the mystery. There are also mini-games and an inventory system to keep things interesting."

(No loop order, no costs, and two orphan features glued on.)

■ **Good:** "1. Choose a suspect. 2. Ask a free-form question (costs 1 action). 3. The notebook records claims and flags contradictions. 4. Press a suspect with a Lead (costs 1 action; wrong evidence hardens them).

5. Accuse — once, ending the game."

(Numbered, costed, and the press step has a failure consequence.)

4. Player Experience (What the Player Sees)

Purpose: The screen, described so a UI can be built from prose alone. This section exists because the #1 GDD failure (per the Class 03 agent review) is describing what the *system* computes instead of what the

player experiences.

Must contain:

- **Every element on the main screen, named:** panels, meters, trays, indicators — each with what it shows and when it updates.
- **The feedback loop of information:** how the player learns they're making progress (highlights, meters, sounds).
- **Moment-to-moment texture:** one or two concrete examples of actual game output the player would read/see, written verbatim.

Must NOT contain: Wireframe theory, color palettes (appendix material), "the UI will be intuitive and clean."

Length: Half a page to 1 page.

Writing rule: every sentence's subject should be *the player* or a *named screen element*. If a sentence's subject is "the system," "the backend," or "the model," it belongs in §5 or §6.

5. AI / Agent Architecture

Purpose: Define each agent as a worker with a job description, and prove the agents produce gameplay, not just text. This is the course's signature section, graded directly by the Agent Role Clarity rubric line.

Must contain — per agent, all four (the course's mandated pattern):

Field	Rule
Name	A proper noun ("Consistency Warden Agent"), not "the checker"
Role sentence	One plain-English sentence: what it produces, when it runs
Gameplay effect	What the <i>player</i> experiences because this agent exists
Output format	Exact JSON schema / field list, with limits (max words, enums)

Plus, once for the whole section:

- **An interaction diagram** — who feeds whom (ASCII art is fine).
- **When each agent runs** — pre-game, per turn, on event.
- **The anti-hallucination story** — what stops agents from breaking the game's logic (ground-truth file, validator agent, knowledge slicing).

Must NOT contain:

- Agents "described by vibe, not by output format" (Class 03 red flag).
- More agent types than the team can build — ****5+ agent types for a solo developer is the canonical scope-creep red flag.****
- Abstract multi-agent vocabulary ("emergent swarm coordination") unless it explains something specific about *this* game's agents.

Length: 1–2 pages. Roughly a quarter of the document.

■ **Bad:** "A network of intelligent agents collaborates dynamically to create emergent narrative experiences tailored to each player."

(Zero named agents, zero formats, pure vibe. This is the anti-slop gate.)

■ **Good: "Notebook Agent** — extracts factual claims from each reply and detects contradictions between them. *Gameplay effect:* mints the Leads the player spends in a Press. *Output:* {new_facts[] {suspect_id, claim, turn_ref}, contradiction|null {fact_a, fact_b, lead_text}}."

6. Technical Strategy

Purpose: Prove the game can actually be built and run at acceptable cost and speed, by this team, with these tools.

Must contain:

- **Stack, in one short list** — language, framework, API, frontend. No justification essays; one line of "why" per choice at most.
- **Model assignment** — which model runs which agent, and why (cost/quality trade stated in one sentence).
- **The token budget as a table** — per agent: calls per session, tokens in/out per call, session total. Plus a cost-per-playthrough estimate and a **hard cap with a defined in-game failure behavior** (what the player sees if the cap hits).
- **Latency plan** — target seconds per player-visible response, and the trick that covers the gap (parallel calls, typing indicators).
- **At least one named constraint** and the design decision it caused.

Format: *"Constraint: X. Therefore the design does Y."* This sentence pattern is graded directly (Scope Realism / Technical Feasibility).

Must NOT contain: Code, class diagrams, database schemas, deployment plans, speculative scaling ("if we reach 1M users").

Length: 1 page. The token table does most of the talking.

7. Scope & Development Plan

Purpose: Kill scope creep in writing, before the agent review crew does it for you.

Must contain:

- **The shipped-scope list** — exactly what's in the build ("1 case, 5 suspects, 6 evidence items, 30-action night. Nothing else."). The closing "Nothing else." is load-bearing: it converts a list into a boundary.
- **A week-by-week plan** for the real remaining time, one line per week, each week ending in something runnable.
- **The first cut** — name the feature you'll drop first if behind. Deciding this now is cheap; deciding it in week 4 is a crisis.

Must NOT contain: Stretch goals dressed as features, post-launch roadmaps, "if time permits" lists longer than the core list.

Length: Half a page.

Optional Appendices (only if they change what gets built)

- **Narrative bible** — only for narrative-driven games; keep canon facts in tables, not prose.
- **Balancing tables** — numbers for economies, damage, drop rates.
- **Art/audio direction** — references and constraints, not mood essays.

If an appendix could be deleted without any code changing, delete it.

Part IV — How to Formulate a Paragraph

The house paragraph formula, in order, ≤ 4 sentences:

1. **Claim** — what exists or happens. ("Pressing costs 1 action.")
2. **Mechanism** — how, with the number or format. ("A press attaches one Lead or evidence item to a named suspect.")
3. **Consequence for the player** — the fork. ("A lying suspect cracks and reveals a new fact; an honest one pushes back and hardens by +1 stress.")

4. (Optional) **Boundary** — the limit or failure case. ("Wrong evidence wastes the action.")

Sentence-level rules:

Rule	■ Don't	■ Do
Present tense	"The player will be able to press"	"The player presses"
Active voice	"Suspects can be interrogated"	"The player interrogates suspects"
No modal hedging	"could / might / may / should ideally"	"does / costs / triggers"
Numbers, not vibes	"replies are short"	"replies are ≤ 60 words"
Enumerate, don't gesture	"various tones"	"tone $\in$ {calm, defensive, evasive, hostile, cracking}"
One subject	"The system and the player..."	Split into two sentences

The banned-words list (each is a missing spec — replace with the number or mechanism it's hiding): *immersive, engaging, dynamic, innovative, revolutionary, seamless, intuitive, fun, deep, rich, emergent* (unqualified),

AI-driven (unqualified), *endless possibilities, next-generation, unique*.

You may keep exactly **one** comparison sentence ("X meets Y") in the Executive Summary. Everywhere else, comparisons are a smell: they mean you haven't specified your own mechanic.

Part V — Formatting Rules

- **Headings** follow the skeleton in Part III, numbered. Reviewers navigate by number ("your §5.2 contradicts §3.1").
 - **Tables** for anything with 3+ parallel facts (budgets, agent rosters, rubrics). Prose comparing four numbers is always worse than a table.
 - **Numbered lists** for sequences (loops, plans). **Bullets** only for unordered sets. Never bullet a sequence.
 - **Code blocks** for output formats and diagrams. JSON schemas go in backticks, never in prose.
 - **Bold** for the first appearance of every named thing (mechanic, agent, resource). Bold is a definition marker, not emphasis — if it's bold twice, one of them is wrong.
 - **Length target: 4–7 pages** for a course-scale game. Under 3 pages, sections are missing; over 10, you're writing the novel nobody maintains.
 - Ship as **PDF** for submission; keep the **Markdown source in the repo** as the living document (this is what gets revised after each review round).
-

Part VI — The Review Gauntlet (Before You Submit)

Run these in order. Each maps to a Class 03 review-crew agent.

1. The Exploit Hunter pass (math check): For every currency/resource, verify the win condition is reachable: add up what the player can earn vs. what winning costs. The canonical bug: "10 gold per enemy, boss costs 500, gold caps at 200" — mathematically unwinnable. Then try to break it the other way: what's the degenerate strategy that wins without playing (spam one action, farm one loop)? Write down the rule that blocks it.

2. The Narrative Consistency pass: Any fact stated twice must match (§2 says 5 suspects → §5's diagram must show 5). Any character motivation must be compatible with their mechanical behavior.

3. The Pacing & Flow pass: Walk the loop as three player types — optimizer, explorer, casual. Where does each get stuck, bored, or frustrated? Is there a dead zone where no action makes progress?

4. The Scope pass: Count your agent types (>4 for a solo dev: cut). Count weeks vs. features (each week must end runnable). Confirm the "first cut" feature is named.

5. The Human pass (agents can't do this one): Give it to one person who plays games but doesn't develop them, and ask exactly three questions:

- "Can you tell me what the player does?" (clarity)
- "Does this sound fun to you?" (the un-automatable question)
- "What's missing?" (fresh eyes)

Then re-read it yourself after 24 hours away. Read the revised sections aloud. Vague when spoken = vague on paper.

6. The rubric pass: Score yourself honestly against the assignment rubric line by line before anyone else does.

Part VII — Quick Self-Audit Checklist

- ☐ A stranger could describe the game after reading §2 alone
- ☐ Win **and** loss conditions stated explicitly
- ☐ Core loop written as a numbered list with per-step costs
- ☐ Every screen element named; every sentence in §4 has player/UI subject
- ☐ Every agent: name + role sentence + gameplay effect + output format
- ☐ ≤ 4 agent types (solo dev)
- ☐ Token budget table + cost estimate + hard cap with player-visible fallback
- ☐ At least one "Constraint: X, therefore Y" sentence
- ☐ Shipped-scope list ends with "Nothing else."
- ☐ Week-by-week plan; "first cut" feature named
- ☐ Zero banned words; zero "will be able to"; zero unfalsifiable claims

- [] 4–7 pages; PDF for submission, Markdown in repo
 - [] Ran all six review passes
-

Sources & Further Reading

- Tim Ryan, [*The Anatomy of a Design Document, Part 1*](<https://www.gamedeveloper.com/design/the-anatomy-of-a-design-document-part-1-documentation-guidelines-for-the-game-concept-and-proposal>) — the classic two-part anatomy (concept/proposal vs. functional spec)
- [*How to Write a Game Design Document*](<https://www.gamedeveloper.com/design/how-to-write-a-game-design-document>) — Game Developer's modern take; don't copy formats blindly, fit the doc to the game
- [Stone Librande](https://en.wikipedia.org/wiki/Stone_Librande) — the *One-Page Designs* GDC method: diagrams with callouts over prose
- [*How to write a game design document (with examples)*](<https://gamedevbeginner.com/how-to-write-a-game-design-document-with-examples/>) — Game Dev Beginner's practical walkthrough
- [*From GDD Graveyard to Living Document*](<https://www.wayline.io/blog/lean-gdd-game-design-documentation>) — the lean/living-document school
- ELVTR course, Class 03: *From GDD to Prototype* — spec-not-wish rule, agent red flags, review-crew method, revision discipline